

DATA ANALYSIS INTERNSHIP

DAY 8 REPORT

Pandas Data Analysis Operations

Introduction

On Day 6, various **Pandas dataframe operations** were explored using Google Colab. Practical activities included **dataset loading, filtering, statistical calculations, string handling, merging datasets, sorting, and modifying dataframe columns** using student-related data.

1. Importing Dataset

The dataset was imported into Pandas using the Excel reading function.

Function Used:

```
pd.read_excel()
```

Purpose:

This function helps in loading Excel files into a dataframe for further processing and analysis.

2. Exploring Dataset Details

Several dataframe functions were used to inspect dataset information.

Functions Practiced:

- `head()`
- `tail()`
- `shape`
- `columns`
- `dtypes`
- `info()`

Purpose:

These functions provide an overview of the dataset, including:

- Number of rows and columns
 - Column labels
 - Data types of each column
 - Overall dataset structure
-

3. Statistical Calculations

Basic statistical analysis was performed on student subject marks.

Functions Used:

- `sum()`
- `mean()`
- `min()`
- `max()`

Activities Performed:

- Calculated total mathematics score
 - Found average physics score
 - Identified minimum chemistry marks
 - Determined maximum biology marks
-

4. Finding Unique Data

The `unique()` function was applied to the student name column.

Purpose:

Used to display non-repeating values from the dataset.

5. Dataframe Modification

Different methods were practiced to update and modify dataframe columns.

Functions Used:

- `drop()`
- `rename()`
- `astype()`
- `replace()`

Purpose:

These functions are useful for:

- Deleting unwanted columns
 - Changing column names
 - Modifying data types
 - Replacing specific values
-

6. Conditional Filtering

Rows were filtered using conditions to extract required information.

Functions Used:

- `query()`
- Conditional selection

Examples:

- Students scoring above 90 in mathematics
- Students having geography marks equal to 90

Purpose:

Helps in selecting data based on specific conditions.

7. Selecting Columns

Single and multiple columns were retrieved from the dataframe.

Methods Used:

```
df["Student_Name"]
```

```
df[["Student_Name","Id"]]
```

Purpose:

Used for accessing only required columns from the dataset.

8. String Handling Functions

Different text-processing operations were performed on student names.

Functions Practiced:

- `upper()`
- `swapcase()`
- `len()`
- `startswith()`
- `endswith()`

Purpose:

Helpful for formatting text and performing string-related analysis.

9. Range-Based Filtering

The `between()` function was used for filtering history marks.

Purpose:

Selects rows whose values fall within a given range.

10. Importing CSV File

A CSV dataset was loaded for analysis.

Function Used:

`pd.read_csv()`

Purpose:

Used to import CSV files into a Pandas dataframe.

11. Merging Dataframes

Two dataframes were combined using a common key.

Function Used:

```
pd.merge(df, df1, on="Id")
```

Purpose:

Combines multiple datasets using a matching column.

12. Joining Dataframes

The join operation was used to combine datasets side by side.

Function Used:

```
df.join(df1, lsuffix="_left", rsuffix="_right")
```

Purpose:

Adds columns from another dataframe while avoiding duplicate names using suffixes.

13. Concatenating Dataframes

Multiple datasets were merged vertically.

Function Used:

```
pd.concat([df1, df2])
```

Purpose:

Used to append one dataframe below another.

14. Handling Missing Values

Missing values were managed using a fill operation.

Function Used:

```
df.fillna(0)
```

Purpose:

Replaces null or empty values with zero.

15. Sorting Dataset

Sorting operations were applied to organize data.

Function Used:

```
sort_values()
```

Purpose:

Arranges dataframe values in ascending or descending order.

16. Checking Dataset Size

The size attribute was used.

Purpose:

Displays the total number of elements present in the dataframe.

17. Adding a New Column

A new column called **Age** was inserted.

Function Used:

```
df.insert(1, "Age", [20]*20)
```

Purpose:

Adds a new column at a specified location in the dataframe.

18. Updating Columns

The assign function was used to add or modify values.

Function Used:

```
df = df.assign(Age=[25]*20)
```

Purpose:

Allows assigning new values or updating existing columns.

Conclusion

On Day 6, practical knowledge of **Pandas dataframe operations** was improved through hands-on activities. Different techniques such as **loading datasets, filtering records, statistical analysis, text processing, dataframe merging, sorting, null value handling, and column manipulation** were practiced successfully. This session enhanced understanding of **data handling and analysis using Pandas in Python**.